Aircraft Engine Remaining Useful Life Prediction Using Machine Learning

Prateek Shakya, Yogesh Modi, Kapish Rohilla, Abhishek Kumar Patel, and Vikas Kumar
Advanced Machine Learning Project Report

***Abstract*—This project investigates machine-learning methods for estimating the Remaining Useful Life (RUL) of aircraft engines using operating settings and sensor readings from the FD001 subset of the NASA C-MAPSS turbofan engine dataset. The workflow covers data inspection, RUL target construction, exploratory sensor analysis, rolling-window feature engineering, and comparison of linear and tree-based regression models. The recorded validation results show that Random Forest achieved the lowest mean absolute error (MAE), while Random Forest and Gradient Boosting reported the same root mean squared error (RMSE). The results suggest that tree-based models capture some nonlinear sensor relationships better than the linear baselines, although predictions for engines with high RUL remain challenging.**

***Index Terms*—Remaining Useful Life, predictive maintenance, aircraft engines, regression, feature engineering, NASA C-MAPSS**

I. Introduction

Remaining Useful Life (RUL) estimates how many operating cycles remain before an engine reaches its recorded end-of-life point. For example, if an engine's final recorded cycle is 200 and its current cycle is 150, its RUL is $200 - 150 = 50$ cycles. Estimating RUL from sensor measurements can support predictive maintenance by helping identify when an engine may need attention.

The objectives of this project are to prepare and understand the aircraft-engine dataset; calculate training RUL labels; examine sensor behaviour and remove constant or nearly constant columns; create rolling features that summarise recent sensor behaviour; train and compare regression models; and evaluate predictions using MAE and RMSE, including identifying cases where the models struggle.

II. Dataset and Tools

The project uses the FD001 subset of the NASA C-MAPSS turbofan engine dataset. Each row represents one engine at one operating cycle. The original data contains an engine identifier, cycle number, three operating settings, and 21 sensor readings. The tools used are Python, Pandas, NumPy, Matplotlib, scikit-learn, and Jupyter Notebook.

III. Methodology

The workflow consists of six stages:

1) **Load and inspect data:** read the raw files, assign column names, check row counts, and inspect missing values.

TABLE I
Dataset Summary

Dataset	Size	Use
Training data	20,631 rows; 100 engines	Recorded through final cycle; used to calculate training RUL.
Test data	13,096 rows; 100 engines	Engine histories stop before failure.
Test RUL file	100 values	Supplies true RUL at each test engine's last observed cycle for final evaluation.

2) **Create RUL labels:** for each training observation, subtract the current cycle from the final recorded cycle of the same engine.
3) **Explore sensor data:** plot sensor trends, identify flat columns, and examine sensor correlations.
4) **Engineer features:** calculate a 10-cycle rolling mean, standard deviation, and slope for each retained sensor.
5) **Train models:** compare OLS Linear Regression, Elastic Net, Random Forest, and Gradient Boosting.
6) **Evaluate models:** use the same validation points for each model and compare MAE and RMSE on validation and test data.

IV. Data Preparation and Feature Engineering

The original dataset has 26 columns. The training set contains 20,631 rows and the test set contains 13,096 rows; each has 100 engines and no missing values. Adding an RUL column gives the training data 27 columns.

Eight constant or nearly constant columns were identified and excluded from the useful sensor list: `setting_3`, `sensor_1`, `sensor_5`, `sensor_6`, `sensor_10`, `sensor_16`, `sensor_18`, and `sensor_19`. This leaves 14 retained sensors. For each retained sensor, the feature-engineering step calculates the mean, standard deviation, and slope over the latest 10 cycles. Together with the current cycle and raw readings from the 14 sensors, these yield 57 model input features. Rolling calculations use only current and previous readings from the same engine to avoid leaking future readings into a prediction.

V. Machine-Learning Models and Evaluation

Four regression approaches are compared:

- **OLS Linear Regression:** baseline linear model.
- **Elastic Net:** linear regression with regularisation that can shrink or eliminate less useful coefficients.

TABLE II
RECORDED VALIDATION RESULTS

Model	MAE	RMSE
OLS Linear Regression	32.86	42.92
Elastic Net	31.66	41.77
Random Forest	25.03	40.17
Gradient Boosting	26.12	40.17

- **Random Forest:** an ensemble of decision trees that can model nonlinear relationships.
- **Gradient Boosting:** builds trees sequentially, with later trees reducing earlier prediction errors.

MAE is the average absolute difference between predicted and actual RUL, measured in cycles. RMSE also measures prediction error but penalises larger errors more strongly. Lower values indicate better predictions. The team uses an engine-based split, with 80 engines for training and 20 for validation, to reduce leakage risk. Five validation cut-off points are used for each validation engine, giving 100 validation points.

VI. RESULTS AND DISCUSSION

The recorded data-preparation and linear-model notebooks show that the data can be loaded consistently, RUL labels can be generated, and rolling features contain no missing values. Sensor analysis indicates that some readings remain relatively stable for much of an engine's life and then change as degradation progresses. Random Forest has the lowest reported validation MAE. Random Forest and Gradient Boosting have the same reported RMSE. The tree-based models reduce average absolute error compared with the linear baselines, suggesting that nonlinear models capture sensor patterns that a straight-line model may miss.

The recorded error analysis indicates that all four models struggle when an engine has a very high RUL: predictions tend to remain below the largest true RUL values. One possible explanation is that the 10-cycle rolling features summarise recent behaviour but do not fully represent the engine's longer degradation history. These findings should be interpreted alongside the current validation setup; the report notes that the full final-evaluation notebook still needs to be run to confirm results for every model.

VII. TEAM CONTRIBUTIONS

VIII. CONCLUSION AND FUTURE WORK

This project establishes a machine-learning workflow for estimating aircraft-engine RUL from sensor data. The team prepares the data, creates RUL labels, studies sensor behaviour, generates rolling features, and compares linear and tree-based models using MAE and RMSE. The recorded results suggest that tree-based models perform better than the linear baselines under the current validation setup, although predicting engines with long remaining life remains difficult.

A useful next step is to run the full final-evaluation notebook and confirm results for every model. For a later phase, a sequence model such as a Long Short-Term Memory (LSTM) network could be tested because it can learn from longer sensor histories rather than relying only on fixed summaries of the latest 10 cycles.

REFERENCES

[1] NASA, "C-MAPSS Jet Engine Simulated Data," dataset description and download details. [Online]. Available: https://data.nasa.gov/

[2] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *Proceedings of the International Conference on Prognostics and Health Management*, 2008.

[3] The pandas development team, "Windowing operations," pandas documentation. [Online]. Available: https://pandas.pydata.org/docs/

[4] scikit-learn developers, "Regression metrics," and documentation for Random Forest and Gradient Boosting. [Online]. Available: https://scikit-learn.org/stable/

TABLE III
TEAM MEMBERS AND CONTRIBUTIONS

Enrollment No.	Name	Role / Contribution
2401020048	Prateek Shakya	Data loading and EDA: named columns, calculated training RUL, checked data quality, plotted sensor trends, identified eight flat columns, and checked correlations.
2401010519	Yogesh Modi	Feature engineering: created 10-cycle rolling mean, standard deviation, and slope features for 14 sensors using only current and past readings.
2401010210	Kapish Rohilla	Validation and linear models: prepared the engine-based split and MAE/RMSE metrics; trained and evaluated OLS and Elastic Net.
2401010020	Abhishek Kumar Patel	Tree-based models: trained and compared Random Forest and Gradient Boosting using the prepared features and validation points.
2401010503	Vikas Kumar	Final evaluation: combines predictions, compares validation and test results, analyses early- and late-life errors, and summarises limitations.